



**Politecnico
di Torino**

Politecnico di Torino

Bachelor degree in Management Engineering

A.a. 2025/2026

Graduation Session ... 2026

Permutation flowshop scheduling with periodic maintenance

Supervisor:

Fabio Salassa

Candidate:

Dania Ottenga

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodore sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Tables	VI
List of Figures	VII
1 Introduction to combinatorial optimization	1
1.1 Historical roots	1
1.2 Families of problems	2
1.3 Complexity and Solving Strategies	2
1.3.1 NP-hard Problems	2
1.3.2 Heuristics and Metaheuristics	2
2 Introduction to Machine Scheduling	5
2.1 Problem classification	5
2.2 Flow shop scheduling	6
2.3 Permutation Flow Shop (PFSSP)	6
3 Problem identification	7
3.1 Mathematical Notation: Parameters, Indexes and Variables	7
3.2 Constraints and objective function	7
3.3 Modeling Maintenance in Idle Times	9
3.4 The Proposed Metaheuristic	9
4 Procedures	11
4.1 Data Management and Input	11
4.2 MIP Implementation	12
4.3 Metaheuristic Implementation	12
4.3.1 Schedule Operation	12
4.3.2 Compute Schedule	14
4.3.3 Makespan	14
4.3.4 Swap Move	16
4.3.5 Insertion Move	16

4.3.6	Get Neighbour	16
4.3.7	Simulated Annealing	16
4.4	Results Visualization: The Gantt Chart Generator	19
4.5	Automation of the Implementation	19
5	Results	21
5.1	Analysis of the obj. values	23
5.2	Computational Time Analysis	23
5.3	% Gap Analysis	25
5.4	Gantt Charts Analysis	27
6	Conclusions	31
A	Scripts	33
A.1	Data Generation Script	33
A.2	MIP implementation Script	34
A.3	Simulated Annealing Script	40
A.4	Automation Script	48

List of Tables

3.1	Mathematical Notation: Parameters, Indexes and Variables	9
5.1	Computational results: Exact Algorithm vs. Metaheuristic.	21

List of Figures

4.1	Representation of the solution to the problem with the Gantt Chart	19
5.1	Representation of the objective values in an histogram	24
5.2	Representation of the objective values in an histogram	25
5.3	Representation of the computational times in a line graph	26
5.4	Representation of the percentage gaps in a line graph	26
5.5	Gantt representation of instance 43 solved by the metaheuristic . .	27
5.6	Gantt representation of instance 43 solved by the MIP solver	27
5.7	Gantt representation of instance 40 solved by the metaheuristic . .	28
5.8	Gantt representation of instance 40 solved by the MIP solver	28
5.9	Gantt representation of instance 45 solved by the metaheuristic . .	29

Chapter 1

Introduction to combinatorial optimization

Combinatorial Optimization (CO) is the search for a “best” configuration of a set of discrete variables to achieve some goals in a typically finite domain. To optimize means to minimize or maximize the objective function, respecting the constraints. The solution will be a combination of variables, present in the set of all feasible assignments: the search (or solution) space.

A CO problem is defined by:

- **Set of variables:** $V = \{x_1, x_2, \dots, x_n\}$.
- **Variable domains:** $D = \{D_1, D_2, \dots, D_n\}$ where x_i can take on values ($x_i \in D_i$).
- **Constraints among variables:** Ω is the set of restrictions that limits all the possible combinations of values defining the solution space $S \subseteq D_1 \times \dots \times D_n$.
- **Objective function:** $f : S \rightarrow \mathbb{R}$ assigns a numeric value to each admissible configuration.

1.1 Historical roots

The origins of CO lie in economic problems: the planning and management of operations and the efficient use of resources. It was initially conceived by Kantorovich and Koopmans and was motivated by applications in transportation and transshipment [1]. Soon more technical applications were developed and today we see discrete optimization problems everywhere: portfolio selection, capital budgeting, design of marketing campaigns, investment planning and facility location,

political districting, gene sequencing, classification of plants and animals, the design of new molecules, the determination of ground states, layout of survivable and cost-efficient communication networks, positioning of satellites, sizing of truck fleets, layout of mass transportation systems and scheduling of buses and trains, assignment of workers to jobs such as drivers to buses and airline crew scheduling, design of unbreakable codes, etc. The list of applications is endless; even in areas like sports, archaeology or psychology [2].

1.2 Families of problems

We can divide the CO in five macro-groups: **the assignment problems** that aim to find the permutation where the sum of the costs of all the variables is the smallest possible by assigning a task to each agent; **the transportation problems** that aim to minimize the cost of transporting a product between several sources and destinations, **the packing/covering problems** that aim to minimize the number of containers used or the value of the object inserted in them; **the traveling salesman problems** that aim to find the order of the given places that the salesman must visit (each location exactly once) traversing the shortest route and **the scheduling problems** that aim to allocate resources to perform a set of tasks over a specific period minimizing the completion time.

1.3 Complexity and Solving Strategies

The main challenge in CO is the size of the search space S . For many problems, S grows factorially or exponentially with the number of variables n .

1.3.1 NP-hard Problems

Visiting all the possibilities in our search space to find the best one is not always computationally possible; these are known as **NP-hard problems** for which no polynomial time algorithm is currently known. This means that as the problem size increases, the time required to find the "best" configuration using exact methods becomes practically impossible, even for modern computers [3].

1.3.2 Heuristics and Metaheuristics

In these cases, we can rely on algorithms that produce an approximately optimal solution by making a trade-off between computational time and solution quality. So, the aim is to find the one solution that is close to the optimal in a reasonable time [4].

Approximate algorithms are divided into two major categories of algorithms. **Heuristics** are problem-specific algorithms designed to find a good solution quickly. They often follow a greedy approach (e.g., in the traveling salesman problem, always moving to the nearest unvisited city). While fast, heuristics often get stuck in local optima - solutions that are better than their neighbors but worse than the global best. On the other hand, **Metaheuristics** are higher-level frameworks that orchestrate one or more heuristics to explore the search space more efficiently. Unlike simple heuristics, metaheuristics include mechanisms to escape local optima (e.g., by allowing temporary "worse" moves or maintaining a population of solutions). Common examples include Genetic Algorithms, Simulated Annealing, and Ant Colony Optimization.

Chapter 2

Introduction to Machine Scheduling

Machine Scheduling aims to perform a number of jobs, each of which consists of a sequence of operations, by using a number of machines. To complete a job, each of its operations must be processed in the order given by the sequence. The processing of an operation requires the use of a particular machine for a given duration, the processing time. The objective is to find a processing order to minimize the makespan, the time at which the last job in the last machine is finished.

A distinction can be made between sequences and schedules. A **sequence** corresponds to an ordering of the operations on each machine that allows the sequence of the operations corresponding to each job. Corresponding to each sequence there is an infinity of feasible **schedules** obtained by specifying the exact starting and finishing time of each operation. A schedule determines for each job a **completion time** in which its last operation is finished [5].

2.1 Problem classification

To classify the various types of machine scheduling problems, the three field notation introduced by **Graham** is used [6] [7]:

- α for the **resources**: we can consider a single machine, parallel identical machines or flow shop environments.
- β for the **properties of tasks**: where we can consider:
 - **Preemption**: not allowed (one cannot stop the task once it has started).
 - **Precedence**: no constraints, strict constraints (finish-start: the successor cannot start before the predecessor finishes) and generalized constraints.

- **Deadlines:** not assumed, imposed on activities and imposed on the project.
- γ for the **criterion** (the objective function) which is to minimize the makespan ($C_{\max}$).

2.2 Flow shop scheduling

Flow Shop is a processing system in which the task sequence of each job is fully specified (chain precedence) and all the jobs visit the work stations in the same order. In a **pure flow shop** each job visits all stages but more generically can have fewer tasks and can skip some stations. But it's assumed that a job visiting a $i' > i$ station cannot go back to the previous one (i). There is also another case: the **reentrant flow shop**. Here jobs can recycle back and be reprocessed at the same station, or sequence of stations, multiple times.

There are many variations to the general flow shop like the **simple flow shop**. Here we can make these considerations: each stage can handle one operation at a time, each task of a job requires a different machine, all jobs are available simultaneously at the start and remain available until the work is finished, each job can be in one place at the time, no preemption is allowed, when a job is finished in one machine, it's immediately available for the next one so there are no delays due to setups or transfer, intermediate storage between machines is unlimited. Then there is the **general job shop**, which is described exactly like the simple flow shop but the machine sequence of different jobs may differ, and the **hybrid (compound or flexible) flow shop** where work station may consists in several functionally identical processors in parallel [8].

2.3 Permutation Flow Shop (PFSSP)

The **Permutation Flow Shop Scheduling Problem** (PFSSP) consists of a set of jobs on a set of machines with the objective of minimizing the makespan. Each job can be processed on one machine at a time and each machine can process one job at a time and the operations can't be preemptable. The sequence in which the jobs are to be processed is the same for each machine.

The PFSSP problem is denoted by the symbols $n/m/P/C_{\max}$ where n represents the number of jobs, m the number of machines, P is the processing time and $C_{\max}$ is the makespan. Based on the considerations exposed, the **PFSSP** model will be used for the case study analysis [9].

Chapter 3

Problem identification

The problem addressed in this study takes into consideration a permutation flowshop scheduling with periodic maintenance. The aim is, given the number of jobs, the number of machines and the processing time of every job on every machine, to minimize the completion time of the final job in the sequence. The production will be carried out within working shifts. To reduce time wasting given by the transfer of duties and the explanation of the work carried out up to the end of the shift, we will consider the end of the shift as a time limit to finish the job that has started so there are no jobs that start in a shift and end in another. When deciding whether to start a job in a shift, if there isn't enough time to complete the job in the current shift, it'll wait and start being worked in the next one.

3.1 Mathematical Notation: Parameters, Indexes and Variables

To transition from the description of the problem to the mathematical model, it is necessary to define the notation used to represent the production environment and the scheduling constraints [10]. It is explained in more detail in the table **3.1**.

3.2 Constraints and objective function

Objective function

$$\min E_{mn} \tag{3.1}$$

Eq. (3.1) states the objective to be minimized (makespan), defined as the completion time in the last machine (m) of the job in the last position (n).

Costraints

$$\sum_{l=1}^n Z_{jl} = 1 \quad 1 \leq j \leq n \quad (3.2)$$

$$\sum_{j=1}^n Z_{jl} = 1 \quad 1 \leq l \leq n \quad (3.3)$$

Eqs. (3.2) force that each job is assigned to only one position in the sequence, while Eqs. (3.3) ensure that each position is occupied by only one job.

$$E_{il} + \sum_{j=1}^n p_{ij} Z_{jl+1} \leq E_{il+1} \quad 1 \leq i \leq m, \quad 1 \leq l \leq n-1 \quad (3.4)$$

$$E_{il} + \sum_{j=1}^n p_{i+1j} Z_{jl} \leq E_{i+1l} \quad 1 \leq i \leq m-1, \quad 1 \leq l \leq n \quad (3.5)$$

Eqs. (3.4) force that, for each machine, two consecutive jobs do not overlap their processing, while Eqs. (3.5) impose that operations of a job in two consecutive machines do not overlap.

$$E_{11} \geq \sum_{j=1}^n p_{1j} Z_{j1} \quad (3.6)$$

Eq. (3.6) computes the completion time of the job scheduled in the first position in the first machine.

$$E_{il} - sT \leq M(1 - \gamma_{ils}) \quad 1 \leq i \leq m \quad 1 \leq l \leq n, \quad 1 \leq s \leq S \quad (3.7)$$

$$E_{il} - \sum_{j=1}^n p_{ij} Z_{jl} + M(1 - \gamma_{ils}) \geq T(s-1) \quad 1 \leq i \leq m, \quad 1 \leq l \leq n, \quad 1 \leq s \leq S \quad (3.8)$$

Sets of Eqs. (3.7) and (3.8) control that each operation starting in a shift also finishes in within this shift.

$$\sum_{s=1}^S \gamma_{ils} = 1 \quad 1 \leq i \leq m, \quad 1 \leq l \leq n \quad (3.9)$$

Finally, the set of Eqs. (3.9) ensures that each operation is scheduled in one shift.

Table 3.1: Mathematical Notation: Parameters, Indexes and Variables

Symbol	Description
<i>Parameters</i>	
n	Number of jobs
m	Number of machines
$p_{i,j}$	Processing time of job j on machine i
T	Length of the shifts
M	Big number
S	Maximal number of shifts
<i>Indexes</i>	
j	Jobs $1 \leq j \leq n$
i	Machines $1 \leq i \leq m$
l	Positions $1 \leq l \leq n$
s	Shifts $1 \leq s \leq S$
<i>Variables</i>	
E_{ij}	Completion time of job in position j on machine i
Z_{jl}	$\begin{cases} 1 & \text{if job } j \text{ is in position } l \\ 0 & \text{otherwise} \end{cases}$
γ_{ils}	$\begin{cases} 1 & \text{if job in position } l \text{ is in shift } s \text{ on machine } i \\ 0 & \text{otherwise} \end{cases}$

3.3 Modeling Maintenance in Idle Times

In this particular study, we consider the **idle time** (the time between the finish of a job on a machine and the start of the next one on the same machine) as a maintenance opportunity. This approach aims to exploit natural gaps in the schedule, performing maintenance without interrupting active processing, thus preserving productivity.

3.4 The Proposed Metaheuristic

Metaheuristics have been established as one of the most practical approaches to simulation optimization. They are designed to tackle complex optimization problems where other optimization methods have failed to be either effective or efficient. A good metaheuristic implementation is likely to provide near optimal

solutions in reasonable computation times [11]. In this study we will apply the **Simulated Annealing** (SA) metaheuristic.

Simulated annealing is a probabilistic method proposed in Kirkpatrick, Gelett and Vecchi (1983) and Cerny (1985) for finding the global minimum of a cost function that may possess several local minima [12]. In condensed matter physics, annealing denotes a physical process in which a solid in a heat bath is heated up by increasing the temperature of the heat bath to a maximum value at which all particles of the solid randomly arrange themselves in the liquid phase, followed by cooling through slowly lowering the temperature of the heat bath. In this way, all particles arrange themselves in the low energy ground state of the corresponding lattice, provided the maximum temperature is sufficiently high and the cooling is carried out sufficiently slowly [13].

Chapter 4

Procedures

4.1 Data Management and Input

To create the inputs values, such as the number of jobs, the number of machines and the processing time for job for machine and calculate the Big-M value, the shift length and the maximal number of shifts using the data retrieved before, a specific script was developed. The function of the script is to create 50 instances, in the folder of the project, to be then applied in the exact model and in the metaheuristic to analyze the differences in the results.

An example of the output file is shown in **Listing 4.1**¹.

```
1 20 6
2 98 3 85 34 54 65 68 69 98 29 56 16 73 47 73 1 91 29 68 92
3 27 20 28 31 21 74 38 24 9 3 59 22 40 99 64 35 14 76 64 100
4 59 25 10 17 47 5 32 67 29 14 88 86 8 96 79 59 51 46 77 92
5 16 65 73 26 80 57 16 54 39 97 95 62 52 8 52 52 20 38 85 44
6 20 60 9 19 48 28 73 52 5 74 41 41 89 59 90 20 20 70 46 61
7 62 45 82 53 10 6 65 59 18 50 15 34 46 15 69 76 55 15 17 10
8 5742 300 20
```

Listing 4.1: Output of the creation file (instance_1.txt).

The **number of jobs** is an integer number randomly generated between 5 and 20, the **number of machines** is randomly generated between 5 and 10 and the **processing times** are randomly generated between 1 and 100. To ensure maximum flexibility and generalizability of the system, the architecture does not assume a predefined unit of measurement for the processing times in advance. The user has the autonomy to freely define the input, expressing it, for instance, in hours or seconds. The system adopts a time-scale agnostic approach, leaving the

¹For the full source code, please refer to Appendix A.1.

responsibility for dimensional consistency entirely to the user's discretion during the configuration phase.

The **Big-M value** is implemented as the sum of the processing times, this allows the constraints to iterate a non-excessive number of times.

The **shift length** is computed as the product of the maximum processing time generated and a random integer between 2 and 5, this allows a non-excessive and non-poor number of jobs to be scheduled in a shift.

The **number of shifts** is calculated as the maximum number between 1 and the ceiling of the division between the big-M value and the shift length, this allows the solver to not iterate an excessive number of times.

4.2 MIP Implementation

The optimal solution will be calculated using the python programming language, in particular the Python-MIP package. **Python-MIP** is a collection of Python tools for the modeling and solution of Mixed-Integer Linear programs (MIPs).

First the values created before are retrieved, then the variables, the objective function and the constraints will be generated, in the same way as they were discussed in the third chapter. Then the model can optimize the function and provide a value. A timer is set on 300 seconds. If the computation exceeds the timer set, there are two possibilities: a feasible solution is found or no solution is found².

4.3 Metaheuristic Implementation

The input retrieving part is the same as the previous one. To initialize the algorithm the **starting temperature** is valued at 1000, the **cooling rate** at 0.9998 and the **maximum number of iterations** at 100000. The functions of the method are defined as follows³.

4.3.1 Schedule Operation

Algorithm 1 schedules a single operation respecting the shifts. First it states the earliest shift in which the job can start. If the job can be scheduled in the earliest shift it will finish also there, otherwise it'll start and finish in the next shift.

²For the full source code, please refer to Appendix A.2.

³For the full source code, please refer to Appendix A.3.

Algorithm 1 Schedule operation

```

1: function SCHEDULEOPERATION(earliest_start, processing_time, T)
2:   shift  $\leftarrow$  Integer(earliest_start/T)
3:   if earliest_start + processing_time  $\leq$  (shift + 1) * T then
4:     return earliest_start + processing_time
5:   else
6:     return (shift + 1) * T + processing_time
7:   end if
8: end function

```

4.3.2 Compute Schedule

Algorithm 2 computes the E matrix given a permutation list. First it selects the number of the job. If the first machine and the first job are chosen, the earliest start is at the beginning (zero). If the first machine but not the first job are chosen, the completion time of the job before in the same machine is taken. If the first job but not the first machine are chosen, the completion time of the first job of the machine before is taken. If neither the first machine or the first job are chosen, the biggest between the completion time of the job before in the same machine and of the same job in the machine before is taken. Knowing now the earliest start, the operation can be scheduled.

Algorithm 2 Compute Schedule

```

1: function COMPUTESCHEDULE(permutation_list)

2:   for  $i \leftarrow 1$  to  $m$  do
3:     for  $l \leftarrow 1$  to  $n$  do

4:        $j \leftarrow \text{permutation\_list}[l]$ 

5:       if  $i = 0$  and  $l = 0$  then
6:          $\text{earliest\_start} \leftarrow 0$ 
7:       else if  $i = 0$  and  $l > 0$  then
8:          $\text{earliest\_start} \leftarrow E_{0,l-1}$ 
9:       else if  $i > 0$  and  $l = 0$  then
10:         $\text{earliest\_start} \leftarrow E_{i-1,0}$ 
11:       else
12:         $\text{earliest\_start} \leftarrow \max(E_{i-1,l}, E_{i,l-1})$ 
13:       end if

14:         $E_{i,l} \leftarrow \text{SCHEDULEOPERATION}(\text{earliest\_start}, p_{i,j}, T)$ 
15:     end for
16:   end for

17:   return  $E$ 
18: end function

```

4.3.3 Makespan

Algorithm 3 gives the total makespan of the job shop.

Algorithm 3 Makespan

```
1: function MAKESPAN(permutation_list)  
2:    $E \leftarrow \text{COMPUTESCHEDULE}(\textit{permutation\_list})$   
3:   return  $E_{m-1,n-1}$   
4: end function
```

4.3.4 Swap Move

Algorithm 4 swaps two random jobs to see if the solution is better or worse.

Algorithm 4 Swap Move

```

1: function SWAPMOVE(permutation_list)

2:   Select two random indexes  $i, j \in \{1, \dots, |permutation\_list|\}$ 
3:    $permutation\_list[i] \leftarrow permutation\_list[j]$ 
4:    $permutation\_list[j] \leftarrow permutation\_list[i]$ 
5:   return permutation_list
6: end function

```

4.3.5 Insertion Move

Algorithm 5 inserts a job in a different position to see if the solution is better or worse.

Algorithm 5 Insertion Move

```

1: function INSERTIONMOVE(permutation_list)

2:   Select two random indexes  $i, j \in \{1, \dots, |permutation\_list|\}$ 
3:    $elem \leftarrow permutation\_list[i]$ 
4:   REMOVEAT(permutation_list,  $i$ )
5:   INSERTAT(permutation_list,  $j$ ,  $elem$ )
6:   return permutation_list
7: end function

```

4.3.6 Get Neighbour

Algorithm 6 takes on a 50/50 possibility the swap or the insertion move.

4.3.7 Simulated Annealing

Algorithm 7 computes the entire algorithm. The initial solution, sorted by descending values of the sum of the processing time for every job, is selected. The best actual solution is the initial solution. Then, iterating for the maximal number of iterations (100000), it creates a neighbour to the solution (using swapping or

Algorithm 6 Get Neighbour

```

1: function GETNEIGHBOUR(permutation_list)
2:   choice  $\leftarrow$  RandomInteger(0, 1)
3:   if choice = 0 then
4:     return SWAPMOVE(permutation_list)
5:   else
6:     return INSERTIONMOVE(permutation_list)
7:   end if
8: end function

```

insert) and calculates the cost difference between the actual and the neighbour cost. If the neighbour is better it is always accepted, else it is accepted with probability $e^{-\Delta/T}$ where Δ is the difference between the neighbour and the current cost. If the current solution is better than the best solution, it is updated. Then geometric cooling is implemented: if the temperature is too low the iteration is stopped.

Algorithm 7 Simulated annealing

```

1: function SIMULATEDANNEALING(temperature,  $\alpha$ , max_iterations)

2:   current_list  $\leftarrow$  jobs  $\{1, \dots, n\}$  sorted in descending order of  $\sum_{i=1}^m p_{ij}$ 
3:   current_cost  $\leftarrow$  MAKESPAN(permutation_list)

4:   best_list  $\leftarrow$  current_list
5:   best_cost  $\leftarrow$  current_cost

6:   for iteration  $\leftarrow$  1 to max_iterations do

7:     neighbour_list  $\leftarrow$  GETNEIGHBOUR(current_list)
8:     neighbour_cost  $\leftarrow$  MAKESPAN(neighbour_list)
9:      $\Delta \leftarrow$  neighbour_cost  $-$  current_cost

10:    if  $\Delta \geq 0$  or RandomNumber  $< e^{-\Delta/T}$  then
11:      current_list  $\leftarrow$  neighbour_list
12:      current_cost  $\leftarrow$  neighbour_cost
13:    end if

14:    if current_cost  $<$  best_cost then
15:      best_list  $\leftarrow$  current_list
16:      best_cost  $\leftarrow$  current_cost
17:    end if

18:    temperature  $\leftarrow$  temperature  $\ast \alpha$ 
19:    if temperature  $< 10^{-3}$  then
20:      break
21:    end if
22:  end for

23:  return best_list, best_cost
24: end function

```

4.4 Results Visualization: The Gantt Chart Generator

For the visualization of the results it was used the python library “matplotlib” to generate a **Gantt Chart**. On the x -axis the time is displayed and on the y -axis the machines are shown. The planning starts from the first machine on top and proceeds downwards to the last. Every job is indicated with the j_n notation and has a specific color to distinguish from the others. The idle times are yellow colored with oblique lines and the text “maintenance”.

In **Figure 4.1** is presented a simple example.

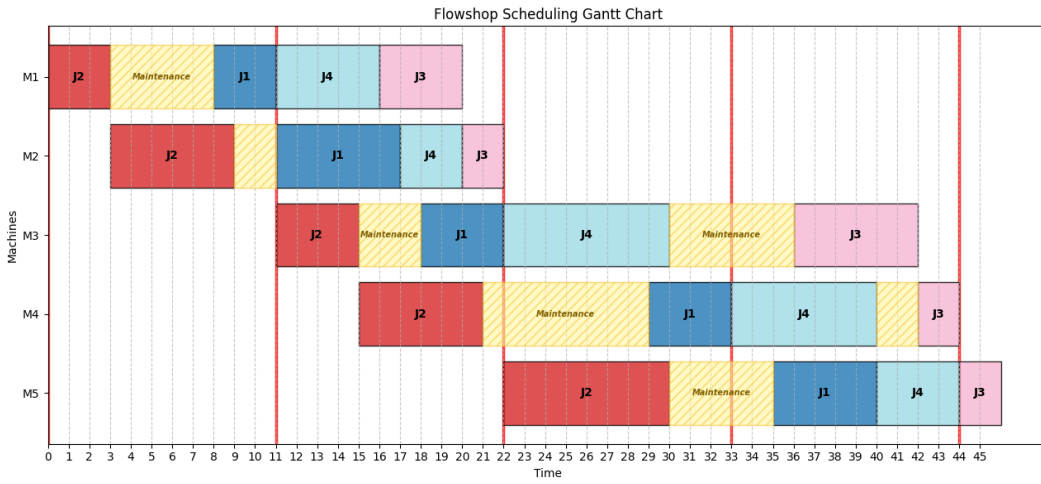


Figure 4.1: Representation of the solution to the problem with the Gantt Chart

4.5 Automation of the Implementation

To implement all of the fifty instances, an automation script was created. The aim is to give the current index to the MIP solver and the metaheuristic, then they will find the related instance created and solve the problem. The results will be saved in a gantt chart format and in a CSV⁴.

⁴For the full source code, please refer to Appendix A.4.

Chapter 5

Results

To rigorously evaluate the performance of the metaheuristic and the exact algorithm, two distinct percentage gaps are computed:

- **Gap vs Best:** Measures the deviation of the metaheuristic solution from the best feasible solution (Upper Bound) found by the exact algorithm.

$$\text{Gap}_{\text{Best}}(\%) = \frac{\text{Obj}_{\text{Meta}} - \text{Obj}_{\text{MIP}}}{\text{Obj}_{\text{MIP}}} \times 100 \quad (5.1)$$

- **Gap vs LB (Worst-case Optimality Gap):** Measures the maximum possible deviation from the theoretical optimum, using the Lower Bound provided by the MIP solver. This metric is particularly useful for instances where the exact algorithm reaches the time limit without proving optimality.

$$\text{Gap}_{\text{LB}}(\%) = \frac{\text{Obj}_{\text{Meta}} - \text{LB}_{\text{MIP}}}{\text{LB}_{\text{MIP}}} \times 100 \quad (5.2)$$

For instances solved to exact optimality within the time limit, the Best Objective and the Lower Bound coincide, rendering the two gap values identical.

All the results are shown in the **Table 5.1**.

Table 5.1: Computational results: Exact Algorithm vs. Metaheuristic.

Inst.	Exact Alg. (MIP)			Metaheuristic		Gaps (%)	
	LB	Best	Obj. Time (s)	Obj.	Time (s)	vs Best	vs LB
1	1254	-	300*	1306	1.5931	-	4.15
2	831	2616	300*	893	0.6779	-65.86	7.46

Continues in the next page...

Table 5.1 – Continuation from the previous page

Inst.	Exact Alg. (MIP)			Metaheuristic		Gaps (%)	
	LB	Best	Obj. Time (s)	Obj.	Time (s)	vs Best	vs LB
3	987	-	300*	1117	1.7283	-	13.17
4	1410	-	300*	1561	1.8332	-	10.71
5	616	616	4.936	616	0.5658	0.0	0.0
6	1392	-	300*	1571	3.3443	-	12.82
7	620	2203	300*	757	0.875	-65.64	22.04
8	751	751	10.949	751	0.6239	0.0	0.0
9	1092	-	300*	1160	1.1247	-	6.23
10	930	-	300*	1075	1.4719	-	15.47
11	922	922	43.2425	922	0.8688	0.0	0.0
12	896	971	300*	965	1.1037	-0.62	7.58
13	865	922	300*	909	0.8424	-1.41	5.09
14	1135	-	300*	1198	1.4361	-	5.55
15	1116	-	300*	1333	3.1144	-	19.44
16	1242	1268	300*	1244	1.16	-1.89	0.08
17	1144	-	300*	1221	1.5781	-	6.73
18	918	918	18.7948	918	0.8507	0.0	0.0
19	954	-	300*	997	1.2218	-	4.4
20	1098	-	300*	1218	1.7468	-	10.93
21	1357	1417	300*	1378	1.2899	-2.75	1.47
22	887	-	300*	957	1.1015	-	7.77
23	1066	1633	300*	1222	1.331	-25.17	14.63
24	589	589	11.8299	589	0.6146	0.0	0.0
25	1206	1409	300*	1265	1.2785	-10.22	4.81
26	749	871	300*	869	0.8982	-0.23	16.02
27	839	839	166.2424	839	0.6534	0.0	0.0
28	786	-	300*	968	0.9055	-	23.16
29	1028	1028	238.0804	1028	0.9796	0.0	0.0
30	639	639	4.6809	639	0.627	0.0	0.0
31	728	854	300*	854	1.3476	0.0	17.31
32	659	659	29.1907	659	0.7066	0.0	0.0
33	939	1089	300*	1089	1.815	0.0	15.97
34	581	581	19.5611	581	0.7594	0.0	0.0
35	985	-	300*	1085	1.8156	-	10.15
36	759	1173	300*	845	1.1847	-27.96	11.33
37	1216	-	300*	1405	2.5387	-	15.48
38	1023	-	300*	1226	1.6332	-	19.73
39	1107	1255	300*	1132	1.8563	-9.8	2.26
40	714	714	40.4761	714	0.9878	0.0	0.0
41	696	799	300*	731	0.9773	-8.51	4.88
42	925	-	300*	1116	1.6181	-	20.52
43	1036	4078	300*	1068	1.3578	-73.81	2.99
44	866	-	300*	988	1.5818	-	14.09
45	1418	-	300*	1690	3.4597	-	19.18

Continues in the next page...

Table 5.1 – Continuation from the previous page

Inst.	Exact Alg. (MIP)			Metaheuristic		Gaps (%)	
	LB	Best Obj.	Time (s)	Obj.	Time (s)	vs Best	vs LB
46	764	1025	300*	878	1.1266	-14.34	14.92
47	1018	-	300*	1163	1.8874	-	14.13
48	645	645	8.0216	645	0.6756	0.0	0.0
49	772	1026	300*	879	1.0432	-14.33	13.71
50	908	-	300*	936	2.0093	-	3.08

* Best feasible solution found. Time limit reached.

5.1 Analysis of the obj. values

Histogram 5.1 and **Histogram 5.2** show the results sorted by ascending values of the number of the jobs and of the machines. Here is observed how the objective value of the MIP implementation in the lower valued instances is equal to the metaheuristic value and the lower bound. These are the cases where the 300 seconds timer doesn't run out. As the number of jobs and machines increases, the MIP solver starts to reach the timer more frequently. So the solutions are found more rarely and, if found, are very far from the LB and the metaheuristic ones.

The results show that the exact algorithm frequently reaches the maximum time, set on 300 seconds (exactly **74%** of the times). That means that it gets lost analyzing all the branches of the search tree, sometimes not even finding a feasible solution. Indeed **42%** of the times it doesn't find a solution and **34%** of the times it finds a feasible solution. Instead the metaheuristic reaches the solution with a **100%** success rate within few seconds.

Furthermore, when the MIP algorithm finds the best solution (within the 300 seconds, the **24%** of the times) the objective value is always the same as the metaheuristic one implements.

The LB value moreover is always equal or less than the optimal solution and the metaheuristic solution. But the LB is just a minimal theoretical value that the function could reach if it was given more time. It is not the actual optimal solution. That's why the metaheuristic could have actually given the optimal solution without knowing it.

5.2 Computational Time Analysis

Line graph 5.3 shows an analysis on the computational time reached by the MIP solver and the metaheuristic.

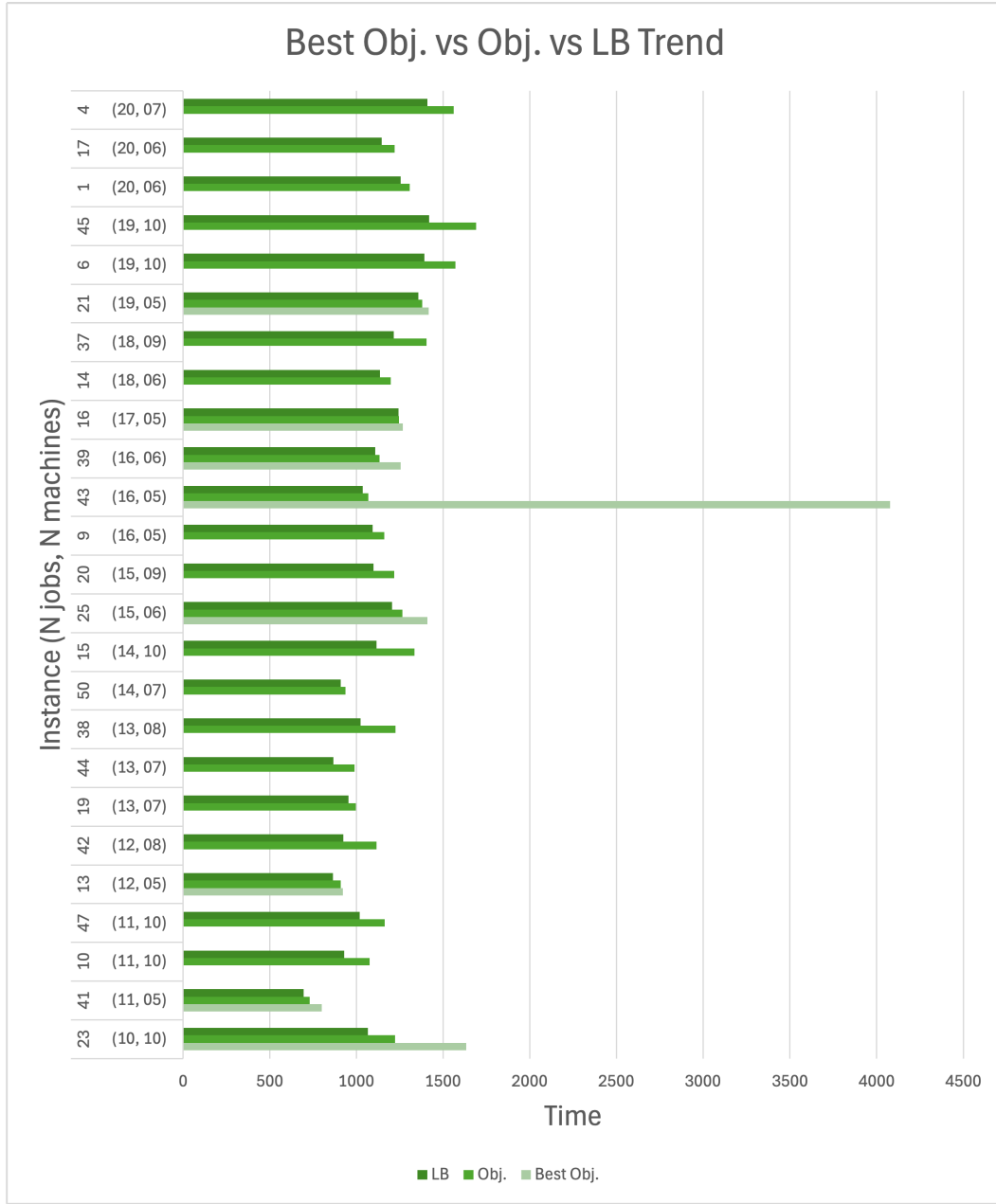


Figure 5.1: Representation of the objective values in an histogram

It is clearly visible that, as the number of jobs and of machines grows, the metaheuristic has a very small variation of the time spent while the MIP solver reaches the 300 seconds very rapidly.

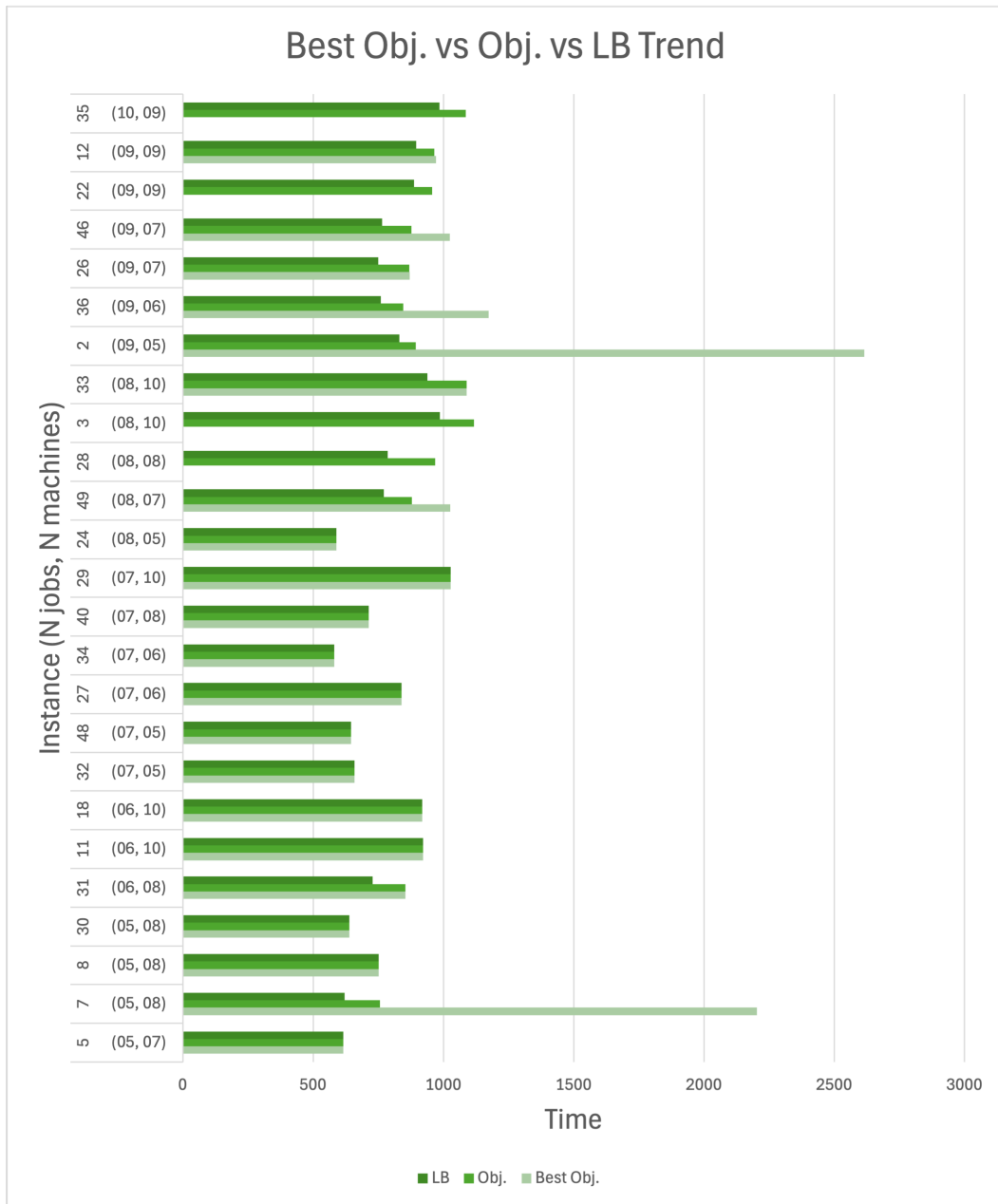


Figure 5.2: Representation of the objective values in an histogram

5.3 % Gap Analysis

Line graph 5.4 shows an analysis on the percentage gaps between the Obj. and the Best Obj. and between the LB and the Obj.

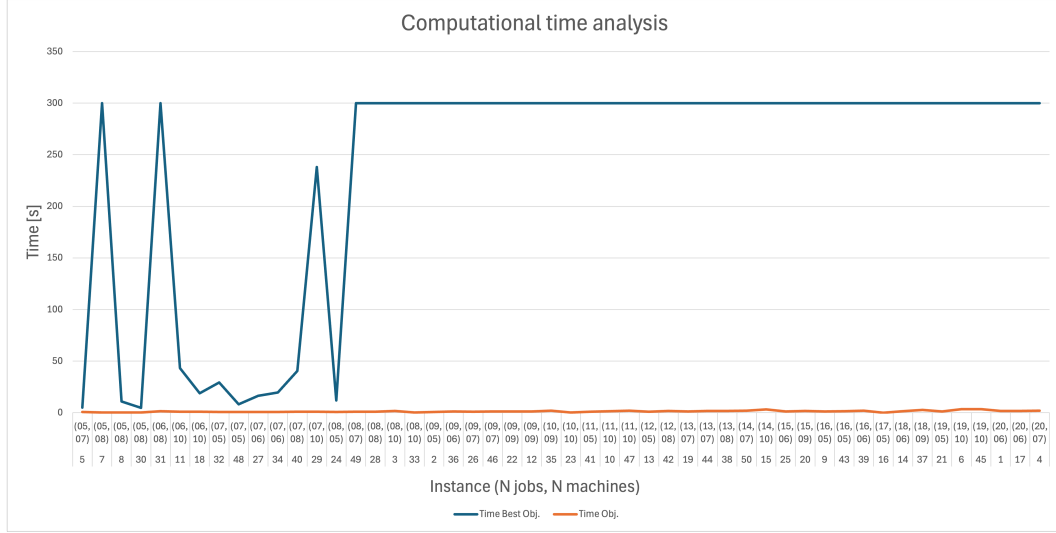


Figure 5.3: Representation of the computational times in a line graph

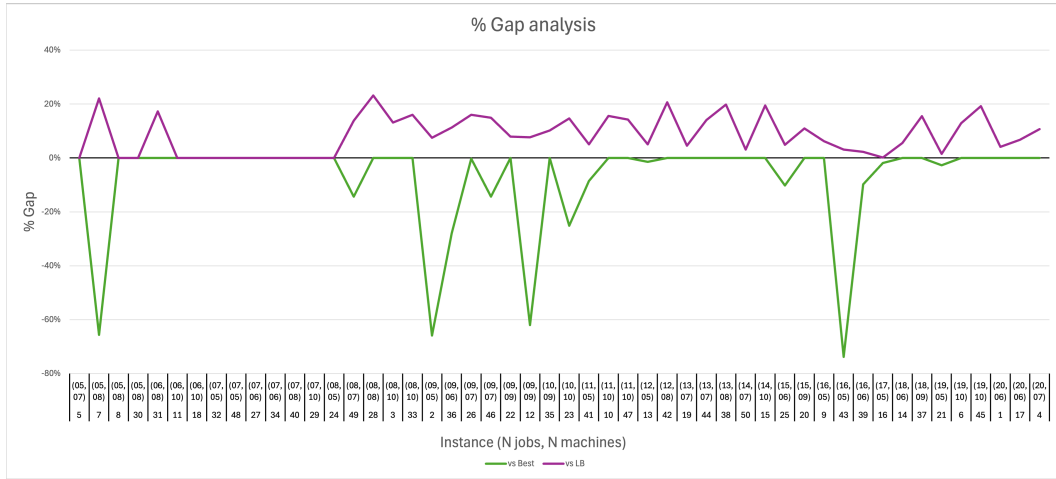


Figure 5.4: Representation of the percentage gaps in a line graph

In the points where the lines coincide on the zero the MIP solver has reached the optimum in the 300 seconds timer. While where the "vs Best" reaches the zero value but the "vs LB" doesn't, represent the instances in which the MIP solver couldn't find a feasible solution.

The "vs LB" indicates that the percentage is always positive while the "vs Best" is always negative. That means that the LB value is always better or equals to the Obj. value of the metaheuristic while the Best Obj. value of the MIP solver is always equal or worse to the Obj. value of the metaheuristic.

5.4 Gantt Charts Analysis

In this section some gantt charts will be compared to show visually the differences pointed out before.

In **Figures 5.5 and 5.6** are shown the charts of an instance with exactly 16 jobs and 5 machines (instance 43). We can see clearly how the MIP solver, when the timer went off, was still trying to analyze all the branches, even the worst ones (like the one given as feasible solution). The difference between the two makespans is the most significant between all the instances.

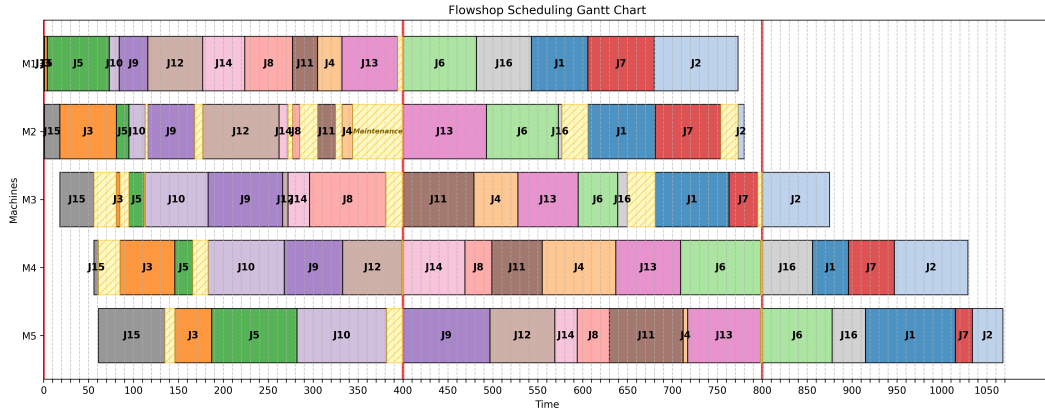


Figure 5.5: Gantt representation of instance 43 solved by the metaheuristic

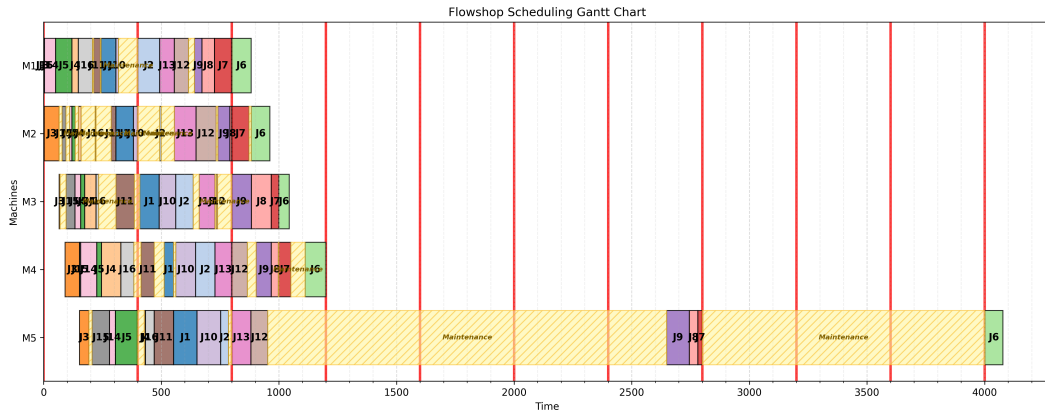


Figure 5.6: Gantt representation of instance 43 solved by the MIP solver

In **Figures 5.7 and 5.8** are shown the charts of an instance simple enough to let the MIP solver find a solution. The makespan of the metaheuristic and of the exact solver are the same but the positions of the jobs are not. This means

that there isn't only one optimal solution or a single optimal job order. Here in particular, four of the seven jobs have different order, but the makespan is in both solutions optimal.

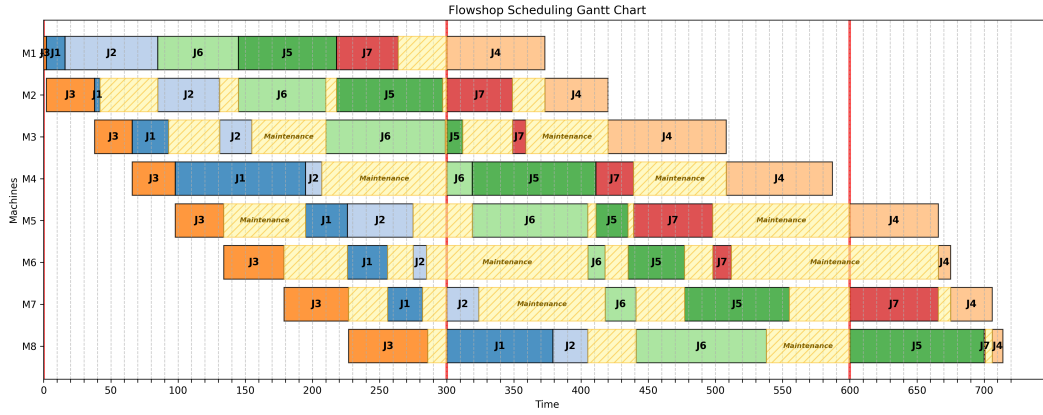


Figure 5.7: Gantt representation of instance 40 solved by the metaheuristic

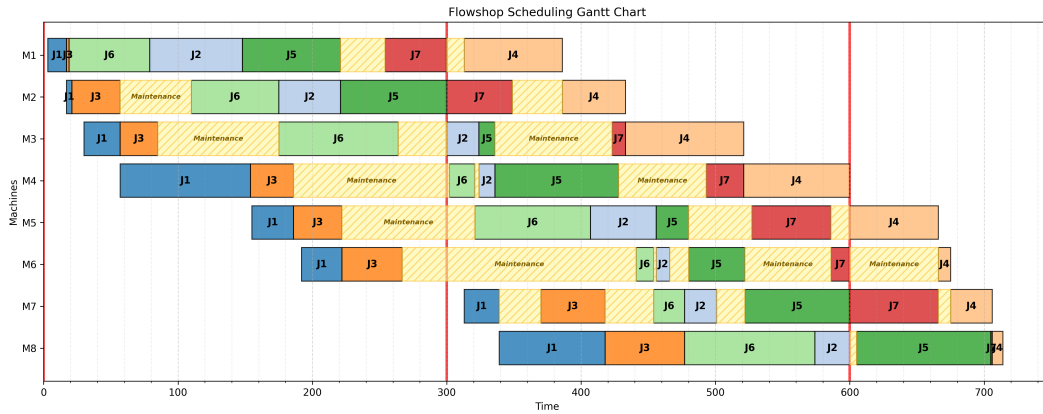


Figure 5.8: Gantt representation of instance 40 solved by the MIP solver

In **Figure 5.9** is shown one of the most complex instances (19 jobs and 10 machines). No solution was found in the MIP solver in the timer set, and the reason is clearly visible from the density and complexity of the solution given by the metaheuristic.

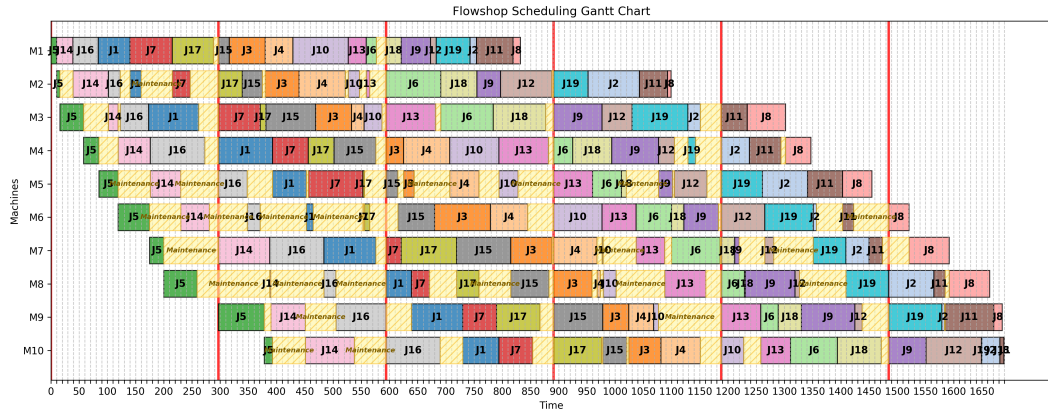


Figure 5.9: Gantt representation of instance 45 solved by the metaheuristic

Chapter 6

Conclusions

Appendix A

Scripts

A.1 Data Generation Script

```
1 import math
2 import os
3 import random
4
5
6 def creation_files():
7
8     directory = os.path.dirname(os.path.abspath(__file__))
9     folder_name = "instances"
10    folder_path = os.path.join(directory, folder_name)
11
12    if not os.path.exists(folder_path):
13        os.makedirs(folder_path)
14
15    for i in range(1, 51):
16        job = random.randint(5, 20)
17        machine = random.randint(5, 10)
18
19        file_name = f"instance_{i}.txt"
20        path = os.path.join(folder_path, file_name)
21
22        with open(path, "w", encoding="utf-8") as f:
23            f.write(f"{job} {machine}\n")
24
25            # Big number
26            M = 0
27            max_p = 0
28            for _ in range(machine):
```



```

29         processing_times = [str(random.randint(1,
100)) for _ in range(job)]
30         for p in processing_times:
31             M += int(p)
32             if int(p) > max_p:
33                 max_p = int(p)
34             f.write(" ".join(processing_times) + "\n")
35
36         # Shift length
37         T = max_p * random.choice([2, 3, 4, 5])
38
39         # Maximal number of shifts
40         S = max(1, math.ceil(M / T))
41
42         f.write(f"{M} {T} {S}")
43
44
45 creation_files()
46 print("The files have been created!")

```

scripts/creation_files.py

A.2 MIP implementation Script

```

1 import os
2 import time
3 from itertools import product
4
5 from matplotlib import ticker
6 from mip import *
7 import matplotlib.pyplot as plt
8 import pandas as pd
9
10 def function(number):
11
12     # =====
13     ''' PARAMETERS '''
14     # =====
15
16     p = []
17     folder_name = "instances"
18     file_name = f"instance_{number}.txt"
19     file_path = os.path.join(folder_name, file_name)

```

```

20     file = f"instance_{number}.txt"
21
22     if os.path.exists(file_path):
23         print(f"File '{file}' loaded succesfully!")
24         with open(file_path, 'r', encoding = 'utf-8') as f:
25             lines = f.readlines()
26             n_lines = len(lines)
27
28             for i, line in enumerate(lines):
29                 if i == 0:
30                     values = line.split()
31
32                     # Number of jobs
33                     n = int(values[0])
34
35                     # Number of machines
36                     m = int(values[1])
37
38                 elif i == n_lines - 1:
39                     values = line.split()
40                     M = int(values[0])
41                     T = int(values[1])
42                     S = int(values[2])
43
44                 else:
45                     values = line.split()
46
47                     # Processing time of job j in machine i
48                     (p ij)
49
50                     l = []
51                     for j in range(len(values)):
52                         l.append(int(values[j]))
53                     p.append(l)
54
55                 else:
56                     print(f"Error: File '{file}' doesn't exist.")
57                     exit()
58
59     mod = Model("Permutation flowshop scheduling") # sense =
60     MINIMIZE
61
62     # =====
63     ''' VARIABLES '''
64     # =====

```

```

63     # Completion time of job in machine i in position l (E
il)
64     E = [[mod.add_var(name="E({},{})".format(i, l), var_type
        = INTEGER)
65             for l in range(n)]
66             for i in range(m)]
67
68     # If job j is scheduled in position l (Z jl)
69     Z = [[mod.add_var(name="Z({},{})".format(j, l), var_type
        =BINARY)
70             for l in range(n)]
71             for j in range(n)]
72
73     # If job in position l is scheduled in the shift s in
machine i (gamma ils)
74     gamma = [[mod.add_var(name="gamma({},{},{})".format(i,
        l, s), var_type=BINARY)
75             for s in range(S)]
76             for l in range(n)]
77             for i in range(m)]
78
79     # =====
80     ''' MODEL '''
81     # =====
82
83     # Objective function
84     mod.objective = minimize(E[m - 1][n - 1])
85
86     # Each job assigned to one position
87     for j in range(n):
88         mod += xsum(Z[j][l] for l in range(n)) == 1
89
90     # Each position assigned to one job
91     for l in range(n):
92         mod += xsum(Z[j][l] for j in range(n)) == 1
93
94     # For each machine two consecutive jobs don't overlap
their processes
95     for (i, l) in product (range(m), range(n - 1)):
96         mod += E[i][l] + xsum(p[i][j] * Z[j][l + 1] for j in
        range(n)) <= E[i][l + 1]
97
98     # Operations of a job in two consecutive machines do not
overlap
99     for (i, l) in product (range(m - 1), range(n)):

```

```

100     mod += E[i][1] + xsum(p[i + 1][j] * Z[j][1] for j in
      range(n)) <= E[i + 1][1]
101
102     # Completion time of the job scheduled in the first
position in the first machine
103     mod += E[0][0] >= xsum(p[0][j] * Z[j][0] for j in range(
      n))
104
105     # Each operation that starts in a shift finishes in the
same
106     for (i, l, s) in product (range(m), range(n), range(S)):
107         mod += E[i][1] - (s + 1) * T <= M * (1 - gamma[i][1
      ][s])
108
109         mod += E[i][1] - xsum(p[i][j] * Z[j][1] for j in
      range(n)) + M * (1 - gamma[i][1][s]) >= T * s
110
111     # Each operation scheduled in one shift
112     for (i, l) in product (range(m), range(n)):
113         mod += xsum(gamma[i][1][s] for s in range(S)) == 1
114
115     # =====
116     ''' OPTIMIZATION '''
117     # =====
118
119     mod.emphasis = SearchEmphasis.FEASIBILITY
120     mod.max_gap = 0.00
121     start = time.time()
122     status = mod.optimize(max_seconds=300)
123     end = time.time()
124     delta = end - start
125
126     if status == OptimizationStatus.OPTIMAL:
127         print('\nOptimal solution cost {} found'.format(mod.
      objective_value))
128
129     elif status == OptimizationStatus.FEASIBLE:
130         print('\nSol.cost {} found, best possible: {}'.
      format(mod.objective_value, mod.objective_bound))
131
132     elif status == OptimizationStatus.NO_SOLUTION_FOUND:
133         print('\nNo feasible solution found, lower bound is:
      {}'.format(mod.objective_bound))
134

```

```

135     if status == OptimizationStatus.OPTIMAL or status ==
OptimizationStatus.FEASIBLE:
136         print('\nSolution:')
137         for v in mod.vars:
138             if abs(v.x) > 1e-6: # only printing non-zeros
139                 print('{ } : {}'.format(v.name, v.x))
140
141     # =====
142     ''' GANTT CHART '''
143     # =====
144
145     if mod.num_solutions:
146
147         gantt_data = []
148         assignments = {}
149         for l in range(n):
150             for j in range(n):
151                 if Z[j][l].x == 1.0:
152                     assignments[l] = j
153
154         for i in range(m):
155             for l in range(n):
156                 job_idx = assignments[l]
157                 end = E[i][l].x
158                 duration = p[i][job_idx]
159                 start = end - duration
160
161                 gantt_data.append({
162                     "Machine": f"M{i + 1}",
163                     "Start": start,
164                     "End": end,
165                     "Duration": duration,
166                     "Job": f"J{job_idx + 1}"
167                 })
168
169         df = pd.DataFrame(gantt_data)
170         fig, ax = plt.subplots(figsize=(15, 6))
171         cmap = plt.get_cmap('tab20')
172         job_colors = {f"J{j+1}": cmap(j) for j in range(n)}
173
174         for i, row in df.iterrows():
175             ax.barh(y=row["Machine"],
176                    width=row["Duration"],
177                    left=row["Start"],
178                    color=job_colors[row["Job"]],

```

```

179         edgecolor='black',
180         alpha=0.8)
181
182         ax.text(x=row["Start"] + row["Duration"] / 2,
183                y=row["Machine"],
184                s=row["Job"],
185                va='center', ha='center', color='black',
fontweight='bold')
186
187         max_time = int(df["End"].max())
188         ax.set_xlim(0, max_time + (max_time * 0.05))
189         ax.xaxis.set_major_locator(ticker.MaxNLocator(nbins=
'auto', integer=True))
190         ax.xaxis.set_minor_locator(ticker.AutoMinorLocator()
)
191         ax.grid(True, axis='x', which='major', linestyle='-',
, alpha=0.5)
192         ax.grid(True, axis='x', which='minor', linestyle='--
', alpha=0.2)
193         ax.set_xlabel('Time')
194         ax.set_ylabel('Machines')
195         ax.set_title('Flowshop Scheduling Gantt Chart')
196         ax.grid(True, axis='x', linestyle='--')
197
198         for s in range(S + 1):
199             shift_position = s * T
200             if shift_position < max_time:
201                 ax.axvline(x=shift_position,
202                           color='red',
203                           linestyle='-',
204                           linewidth=2.5,
205                           alpha=0.8,
206                           zorder=0)
207
208         for m_id in df["Machine"].unique():
209             m_df = df[df["Machine"] == m_id].sort_values("
Start")
210
211             for i in range(len(m_df) - 1):
212                 actual_end = m_df.iloc[i]["End"]
213                 next_start = m_df.iloc[i + 1]["Start"]
214                 duration_gap = next_start - actual_end
215
216                 if duration_gap > 1e-3:
217                     ax.barh(y=m_id,

```

```

218         width=duration_gap,
219         left=actual_end,
220         color='#FFF59D',
221         hatch='///',
222         edgecolor='#FBC02D',
223         alpha=0.6,
224         zorder=2)
225
226         if duration_gap > 50:
227             ax.text(x=actual_end + duration_gap
228 / 2,
229
230                     y=m_id,
231                     s="Maintenance",
232                     va='center', ha='center',
233                     color='#7F6000',
234                     fontsize=7,
235                     fontweight='bold',
236                     fontstyle='italic')
237
238     ax.invert_yaxis()
239     plt.tight_layout()
240
241     folder_name = "gantt_results"
242     image_name = f"gantt_{number}mip.png"
243
244     if not os.path.exists(folder_name):
245         os.makedirs(folder_name)
246
247     path = os.path.join(folder_name, image_name)
248
249     plt.savefig(path, dpi=300, bbox_inches='tight')
250
251 else:
252     print('No solution found')
253
254 return mod.objective_value, mod.objective_bound, delta

```

scripts/main.py

A.3 Simulated Annealing Script

```

1 import math
2 import os

```

```

3 import copy
4 import random
5 import time
6 import pandas as pd
7 from matplotlib import pyplot as plt
8
9 def simulated_annealing(number):
10
11     # =====
12     ''' PARAMETERS '''
13     # =====
14
15     p = []
16     folder_name = "instances"
17     file_name = f"instance_{number}.txt"
18     file_path = os.path.join(folder_name, file_name)
19     file = f"instance_{number}.txt"
20
21     if os.path.exists(file_path):
22         print(f"File '{file}' loaded successfully!")
23         with open(file_path, 'r', encoding = 'utf-8') as f:
24             lines = f.readlines()
25             n_lines = len(lines)
26
27             for i, line in enumerate(lines):
28                 if i == 0:
29                     values = line.split()
30
31                     # Number of jobs
32                     n = int(values[0])
33
34                     # Number of machines
35                     m = int(values[1])
36
37                 elif i == n_lines - 1:
38                     values = line.split()
39                     M = int(values[0])
40                     T = int(values[1])
41                     S = int(values[2])
42
43                 else:
44                     values = line.split()
45
46                     # Processing time of job j in machine i
47
48     (p ij)

```



```

47         l = []
48         for j in range(len(values)):
49             l.append(int(values[j]))
50         p.append(l)
51
52     else:
53         print(f"Error: File '{file}' doesn't exist.")
54         exit()
55
56     # =====
57     ''' FUNCTIONS '''
58     # =====
59
60     def schedule_operation(earliest_start, processing_time,
61 T):
62         # Earliest shift in which the job can start (
starting with 0)
63         shift = int(earliest_start // T)
64
65         # If the job can be scheduled in the earliest shift
it will finish also there
66         if earliest_start + processing_time <= (shift + 1) *
T:
67             return earliest_start + processing_time
68
69         # Otherwise it'll start and finish in the next shift
70         else:
71             return (shift + 1) * T + processing_time
72
73
74     # It computes the E matrix given a permutation list
75     def compute_schedule(permutation_list):
76
77         # Completion time of job in machine i in position l
(E[i][l])
78         E = [[0.0] * n for _ in range(m)]
79
80         for i in range(m):
81             for l in range(n):
82                 # Number of the job
83                 j = permutation_list[l]
84
85                 # If it's the first machine and the first
job

```

```

86         if i == 0 and l == 0:
87             earliest_start = 0
88
89             # If it's the first machine but not the
first job we take the completion time of the job before
in the same machine
90             elif i == 0 and l > 0:
91                 earliest_start = E[0][l - 1]
92
93             # If it's the first job but not the first
machine we take the completion time of the first job of
the machine before
94             elif i > 0 and l == 0:
95                 earliest_start = E[i - 1][0]
96
97             # If it's not the first machine and not the
first job we take the biggest between the completion time
of the job before in the same machine and of the same
job in the machine before
98             else:
99                 earliest_start = max(E[i - 1][l], E[i][l
100 - 1])
101
102             # Knowing now the earliest start we schedule
the operation
103             E[i][l] = schedule_operation(earliest_start,
104 p[i][j], T)
105
106         return E
107
108     # It gives the total makespan of the job shop
109     def makespan(permutation_list):
110         E = compute_schedule(permutation_list)
111         return E[m - 1][n - 1]
112
113     # It swaps two jobs to see if the solution is better or
worse
114     def swap_move(permutation_list):
115         copy_list = copy.deepcopy(permutation_list)
116
117         # We select two positions of the list swapping
118         positions_swapping = random.sample(range(len(
copy_list)), 2)

```

```

119         # And we swap
120         copy_list[positions_swapping[0]], copy_list[
121         positions_swapping[1]] =(
122             copy_list[positions_swapping[1]], copy_list[
123             positions_swapping[0]])
124         return copy_list
125
126     # It inserts a job in a different position to see if the
127     solution is better or worse
128     def insertion_move(permutation_list):
129         copy_list = copy.deepcopy(permutation_list)
130
131         # We select two positions of the list inserting
132         positions_insert = random.sample(range(len(copy_list)
133         )), 2)
134
135         # And we insert
136         elem = copy_list.pop(positions_insert[0])
137         copy_list.insert(positions_insert[1], elem)
138         return copy_list
139
140     # It takes on a 50/50 possibility the swap or the
141     insertion move
142     def get_neighbour(permutation_list):
143         choice = random.randint(0, 1)
144         if choice == 0:
145             return swap_move(permutation_list)
146         else:
147             return insertion_move(permutation_list)
148
149     def simulated_annealing(temperature, alpha,
150     max_iterations):
151
152         # We select the initial solution sorted by
153         descending values of the sum of the processing time for
154         every job
155         current_list = sorted(range(n), key=lambda j: sum(p[
156         i][j] for i in range(m)), reverse=True)
157         current_cost = makespan(current_list)
158
159         # The best actual solution is the initial solution

```

```

155     best_list = current_list[:]
156     best_cost = current_cost
157
158     for iteration in range(max_iterations):
159
160         # Creates a neighbour to the solution using
161         swapping or insert
162         neighbour_list = get_neighbour(current_list)
163         neighbour_cost = makespan(neighbour_list)
164
165         # Cost difference between the actual and the
166         neighbour cost
167         delta = neighbour_cost - current_cost
168
169         # If the neighbour is better it is always
170         accepted, else it is accepted with probability  $e^{(-\text{delta}/T)}$ 
171         if delta <= 0 or random.random() < math.exp(-
172         delta / temperature):
173             current_list = neighbour_list
174             current_cost = neighbour_cost
175
176         # If the current solution is better than the
177         best solution it is updated
178         if current_cost < best_cost:
179             best_list = current_list[:]
180             best_cost = current_cost
181
182         # Geometric cooling
183         temperature *= alpha
184
185         # If temperature is too low we stop the
186         iteration
187         if temperature < 1e-3:
188             break
189
190     return best_list, best_cost
191
192     # =====
193     ''' ALGORITHM '''
194     # =====
195
196     start = time.time()

```

```

192     best_list, best_cost = simulated_annealing(1000, 0.9998,
193         100000)
194     end = time.time()
195     delta = end - start
196
197     # =====
198     ''' GANTT CHART '''
199     # =====
200
201     E = compute_schedule(best_list)
202     gantt_data = []
203
204     for i in range(m):
205         for l in range(n):
206             job_idx = best_list[l]
207             end = E[i][l]
208             duration = p[i][job_idx]
209             start = end - duration
210
211             gantt_data.append({
212                 "Machine": f"M{i + 1}",
213                 "Start": start,
214                 "End": end,
215                 "Duration": duration,
216                 "Job": f"J{job_idx + 1}"
217             })
218
219     df = pd.DataFrame(gantt_data)
220     fig, ax = plt.subplots(figsize=(15, 6))
221     cmap = plt.get_cmap('tab20')
222     job_colors = {f"J{j+1}": cmap(j) for j in range(n)}
223
224     for i, row in df.iterrows():
225         ax.barh(y=row["Machine"],
226             width=row["Duration"],
227             left=row["Start"],
228             color=job_colors[row["Job"]],
229             edgecolor='black',
230             alpha=0.8)
231
232         ax.text(x=row["Start"] + row["Duration"] / 2,
233             y=row["Machine"],
234             s=row["Job"],
235             va='center', ha='center', color='black',
236             fontweight='bold')

```

```

235 max_time = int(df["End"].max())
236 ticks = list(range(0, max_time + 10, 10))
237 ax.set_xticks(ticks)
238 labels = [str(x) if x % 50 == 0 else "" for x in ticks]
239 ax.set_xticklabels(labels)
240 ax.set_xlabel('Time')
241 ax.set_ylabel('Machines')
242 ax.set_title('Flowshop Scheduling Gantt Chart')
243 ax.grid(True, axis='x', linestyle='--', alpha=0.7)
244
245 for s in range(S + 1):
246     shift_position = s * T
247     if shift_position < max_time:
248         ax.axvline(x=shift_position,
249                   color='red',
250                   linestyle='--',
251                   linewidth=2.5,
252                   alpha=0.8,
253                   zorder=0)
254
255 for m_id in df["Machine"].unique():
256     m_df = df[df["Machine"] == m_id].sort_values("Start"
257 )
258
259     for i in range(len(m_df) - 1):
260         actual_end = m_df.iloc[i]["End"]
261         next_start = m_df.iloc[i + 1]["Start"]
262         duration_gap = next_start - actual_end
263
264         if duration_gap > 1e-3:
265             ax.barh(y=m_id,
266                   width=duration_gap,
267                   left=actual_end,
268                   color='#FFF59D',
269                   hatch='///',
270                   edgecolor='#FBC02D',
271                   alpha=0.6,
272                   zorder=2)
273
274             if duration_gap > 50:
275                 ax.text(x=actual_end + duration_gap / 2,
276                       y=m_id,
277                       s="Maintenance",
278                       va='center', ha='center',

```

```
279         color='#7F6000',
280         fontsize=7,
281         fontweight='bold',
282         fontstyle='italic')
283
284     ax.invert_yaxis()
285     plt.tight_layout()
286
287     folder_name = "gantt_results"
288     image_name = f"gantt_{number}.png"
289
290     if not os.path.exists(folder_name):
291         os.makedirs(folder_name)
292
293     path = os.path.join(folder_name, image_name)
294
295     plt.savefig(path, dpi=300, bbox_inches='tight')
296
297     return best_cost, delta
```

scripts/simulated_annealing.py

A.4 Automation Script

```
1 import os
2 import csv
3 from main import function
4 from simulated_annealing import simulated_annealing
5
6 directory = os.path.dirname(os.path.abspath(__file__))
7 folder_name = "results"
8 folder_path = os.path.join(directory, folder_name)
9
10 if not os.path.exists(folder_path):
11     os.makedirs(folder_path)
12
13 #results
14 csv_file_path = os.path.join(folder_path, "max_time.csv")
15
16 headers = ["Instance", "LB", "Best Obj.", "Time 1", "Obj.",
17            "Time 2", "vs Best", "vs LB"]
18
19 with open(csv_file_path, "w", newline="", encoding="utf-8")
20     as f:
```

```

19     writer = csv.writer(f, delimiter=";")
20
21     writer.writerow(headers)
22     #51
23     for i in range(1, 2):
24         solution, lb, time1 = function(i)
25         metaheuristic, time2 = simulated_annealing(i)
26
27         #if time1 >= 300:
28         #    time1 = "300*"
29         #else:
30         time1 = round(float(time1), 4)
31
32         if solution is not None:
33             vs_best = (metaheuristic - solution) / solution
34             * 100
35             solution = int(solution)
36             vs_best = round(float(vs_best), 2)
37         else:
38             vs_best = "-"
39             solution = "-"
40
41         if lb is not None:
42             vs_lb = (metaheuristic - lb) / lb * 100
43             vs_lb = round(float(vs_lb), 2)
44         else:
45             vs_lb = "-"
46
47         row = [i, int(lb), solution, time1, metaheuristic,
48             round(float(time2), 4),
49             vs_best, vs_lb]
50         writer.writerow(row)

```

scripts/automation.py